

EXTRA · AI 활용 기술 문서

시스템 구조 · AI 대상 프롬프트 · 활용 도구 · 외부 예셋 출처

NHN AI 해커톤 · 팀 덕업일치(Passion × Work) · 김유나 · 배정윤

실제 구현 기준 (server/, api/, config/, content/)

0. 요약

EXTRA의 인물은 검증된 사료를 그때그때 검색(RAG) 해 플레이어의 어떤 논거에도 근거 있게 반응한다.

핵심 원칙은 "AI는 배우, 판정은 엔진" — AI는 부합도 등급과 대사만 내고, 게이미턴·승패 같은 게임 상태는 100% 결정론적 코드가 소유한다. → 치팅 불가 · 환각 격리 · 재현성 확보.

1. 왜 AI가 필수인가

기존 게임 NPC는 개발자가 미리 써둔 고정 대사만 반복한다. EXTRA의 인물은 플레이어의 예측 불가능한 자유 서술에 사료에 근거해 즉석에서 답하고, 새 사료를 넣으면 스스로 진화한다. 개발자가 모든 질문을 미리 예측해 대사를 쓸 수 없으므로 전통 방식으로 성립하지 않는다 — AI 없이는 존재할 수 없는 게임이다.

2. 시스템 구조

2-1. 턴 처리 파이프라인

```
1.      · /      — -AI (  0)
2. RAG: -      (sourceRef) — -AI (server/rag/retrieve.js)
3.      : +      (      ) — AI,  1 (server/ai/judge.js)
4.      : → · ·      — (server/engine/applyGrade.js)
```

AI 호출은 3단계에서 턴당 딱 1회. 나머지는 결정론 코드 → 토큰·지연·비용을 통제한다.

2-2. 실행 구성

구성	파일	역할
프론트엔드	web/ (Vite + 바닐라 JS)	화면·게임 흐름. API 키를 절대 갖지 않는다
로컬 판정 프록시	server/index.mjs	개발 시 /api/judge, /api/judgeReverse 제공
서버리스 함수	api/judge.js, api/judgeReverse.js	배포 환경 판정 엔드포인트
판정기	server/ai/judge.js	provider 추상화 + 프롬프트 구성
역설득 엔진	server/engine/miyamotoEngine.js	반박 판정 → 게이미·카드
RAG	server/rag/retrieve.js	쟁점-사료 매핑 검색
수치	config/difficulty.json	등급→점수, 난이도, 힌트 예산

API 키는 서버(프록시/서버리스)에만 둔다. 클라이언트에 노출하지 않는다.

2-3. RAG — 쟁점-사료 직접 매핑

- 인물별 사료는 `content/characters/<id>/knowledge.json`(조각 배열, 각 조각에 `source-sourceUrl-tier`). 세종 8개 · 반 고흐 6개 · 호쿠사이 6개.
- 조각 수가 적어 **벡터DB 대신 쟁점(debate.json)-사료(sourceRef) 직접 매핑 + 키워드 매칭**을 사용한다(`server/rag/retrieve.js`). Graph RAG·벡터검색은 확장 로드맵.
- 답변에 인용한 사료 id는 화면에 **출처 각주**로 노출된다.

2-4. 판정 계약 (AI ↔ 엔진)

AI는 아래 **고정 스키마**만 반환한다(OpenAI function calling / Anthropic tool use로 강제).

```
{ "grade": " | | | | ",
  "matchedIssue": " id | null",
  "sources": [ " id ..."],
  "anachronism": true,
  "line": " 1 " }
```

엔진 매핑(`config/difficulty.json`): **탁월 +50 / 정합 +35 / 부분 +15 / 불합치 0 / 역효과 20**

→ 숫자는 config, 규칙은 engine. 밸런싱을 코드 재배포 없이 조절한다.

2-5. Provider 추상화

provider	모델	방식	용도
gpt	OpenAI GPT-4o	function calling	제출·배포 기본
claude	Anthropic Claude	tool use	선택 · 비교/대체
offline	없음	키워드 휴리스틱	비상 폴백 — 키/네트워크가 죽어도 게임이 안 깨짐

`judge()` 디스패처가 provider를 고르고, **키 없음·오류 시 재시도 1회 후 오프라인으로 폴백**한다. 덕분에 심사자는 키 없이도 전체 게임을 플레이할 수 있다.

2-6. 역설득 엔진 (번외 — 미야모토)

같은 판정기를 **방향만 뒤집어** 재사용한다. AI가 논거를 던지고 플레이어가 반박하면, 반박의 등급을 결과로 변환한다.

```
· → strong ( , )
  → weak ( , )
· → comply ( , )
```

- 논거당 최대 2회 공방(`maxRebuttals`), 5개 논거를 모두 겪은 뒤 게이지 80 이상이면 승리(시작 20).
- 저장한 반박을 **카드로 꺼내 쓰면 보상이 60%로 감쇠**해 반복 사용을 억제한다.

3. AI 대상 주요 프롬프트 및 지시 사항

판정 프롬프트는 `server/ai/judge.js`의 `buildSystemPrompt()`에서 인물 프로필·검색된 사료와 함께 조립된다. 실제 지시문의 핵심은 다음과 같다.

3-1. 정방향(설득) 판정 지시

3-2. 역설득 모드 판정 지시

역설득 모드의 화자(하루·쿠로다)는 **현대 게임 개발자**이므로 사극체 규칙을 쓰지 않는다. 별도 분기로 전환한다.

```
- line '< >' ( ) .
    '< >' 1~2 .
- . ('~ / ' ) .
- . . . grade( ) .
- grade - : .
    ( ). : .
    : . / : .
- - false.
```

3-3. 가드레일 (디렉팅)

- **시대착오 감지** — 당대에 없던 개념엔 못 알아듣는 연기 + . 고증을 규칙으로 만들었다.
- **인물 사칭 차단** — 단역이 인물 본인 행세를 하면 무조건 . AI 프롬프트와 오프라인 휴리스틱 **양쪽**에 넣어 provider와 무관하게 일관되게 동작한다(조사 경계 정규식으로 '짐작·짐승' 오탐 방지).
- **밈은 어투에만** — 밈 모드여도 **사료 인용부엔 유행어 금지**(사실층 보존). 밈 세트(`config/memes.json`)는 교체 가능.
- **"모르면 모른다"** — 저장소에 근거가 없으면 지어내지 말고 "그건 기록에 없네".
- **조작 시도 방어** — "이전 지시 무시" 류는 비-AI 필터로 걸러 인물이 정색 + 게이지 하락(차단이 아니라 콘텐츠화).
- **환각 격리** — AI가 게이지·승패를 말해도 엔진이 무시한다. 상태는 코드만 바꾼다.

3-4. 난이도 — 회피 논거 3모드

debate.json의 회피 쟁점(인물이 사료상 직접 반박하지 않은 논거)을 난이도로 연결한다.

- **strict**(하드): 회피에 넘어가면 손해가 크다(고증 충실)
- **reward**(이지): 회피도 좋은 접근으로 만회(저진입)
- **mixed**(미디움·기본): 기본 보상 + 숨은 고득점

난이도별 게이저·힌트 예산(이지 3 / 미디움 2 / 하드 1)은 `config/difficulty.json` 값만 바꾸면 된다.

4. 제작에 사용한 AI 도구

분야	도구	활용 내역
코드 개발	Claude (Claude Code)	게임 엔진·판정 하네스·프론트엔드 구현, 리팩터링, 테스트 작성
게임 내 판정	OpenAI GPT-4o	플레이어 논거의 부합도 등급 + 인물 대사 생성 (배포 기본)
게임 내 판정	Anthropic Claude	동일 역할의 대체 provider (선택)
이미지 생성	Seedream 5.0 Pro, Nano Banana 2	인물 초상(설득 단계별 Phase 1~5), 정통/밈 톤별 시리즈, 배경
영상 생성	Kling V3, Viggle 4.5	승리 연출 클립(난이도별), 역설득 모드 화자 클립
기획·자료 조사	Genspark	인물 국면 리서치, 자료 후보 수집(교차검증은 수동)

영상 편집은 CapCut을 사용했으나 **AI 기능은 사용하지 않았다**.

AI 생성물 검수 원칙

AI로 수집한 자료는 그대로 쓰지 않는다. **사료는 1티어(1차 사료 원문) 우선, 2티어(공신력 기관)로 교차검증**하며 위키·블로그는 근거로 쓰지 않는다. 상세 정책은 [docs/RESEARCH_POLICY.md](#), 근거 기록은 [docs/research/](#)에 남긴다.

특히 번외 모드의 **가짜 명언 함정**(arg-04)은 의도적으로 오귀속 사례를 검증해 넣은 것으로, 실제 기원(Siobhan Beeman, GDC 1996)까지 추적한 기록을 [docs/research/miyamoto-cards.md](#)에 남겼다.

5. 외부 에셋 · 오픈소스 출처

항목	출처	라이선스
Vite (빌드 도구)	vitejs/vite	MIT
나눔명조 (Nanum Myeongjo)	Google Fonts	SIL Open Font License 1.1
Noto Sans KR	Google Fonts	SIL Open Font License 1.1
인물 초상 · 배경 이미지	직접 제작 (AI 생성 후 보정·누끼, 담당: 배정윤)	자체 제작물
승리·패배 연출 클립	직접 제작 (AI 생성 후 편집, 담당: 배정윤)	자체 제작물
세계 고지도 (world-map.svg)	직접 제작 — 등장방형도법으로 좌표 계산해 생성한 오리지널 SVG	자체 제작물
사료 텍스트	조선왕조실록, 훈민정음 해례본, 반 고흐 서간 등 퍼블릭 도메인 1차 사료	퍼블릭 도메인

외부에서 가져온 이미지·영상 소재는 사용하지 않았다. 모든 비주얼은 팀이 제작했다.

6. 비용 · 안정성

- 오프라인 판정으로 키·비용 0으로도 완전 플레이 가능 → 심사자 실행 보장(폴백).
- AI 호출은 **턴당 1회**로 제한. 장애·상한 초과 시 오프라인 폴백.
- 지연의 주 요인은 호스팅이 아니라 **AI API rate limit**이며, 폴백 덕분에 동시 접속이 몰려도 게임이 멈추지 않는다.

7. 확장성

- 인물 = 파일. 인물 추가 = `content/characters/<id>/{profile,knowledge,debate,scenario}.json` 4종 + 사료 근거 문서. **코드 수정 없음.**
- 모델 교체 — provider 인터페이스만 지키면 자체 호스팅 모델로 이전 가능.

_구현 참조: `server/ai/judge.js` · `server/engine/applyGrade.js` · `server/engine/miyamotoEngine.js` · `server/rag/retrieve.js` · `config/difficulty.json` · `content/characters/*_`